

Take-home assignment: "AI Adapter Log Analyzer" (up to 3 hours)

The goal is to evaluate your ability to design and implement a small AI agent that uses an LLM as a controlled component in a service architecture, not as a single black-box function.

Estimated time limit: **up to 3 hours** of focused work. Clean design and reasonable trade-offs are more important than exhaustive edge-case coverage.

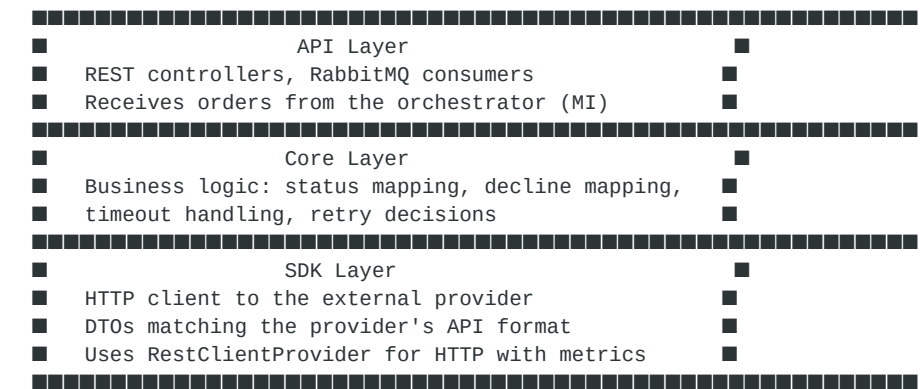
Context

Imagine you are building an internal tool for on-call engineers at a payment processing company. When a payment adapter starts failing, the engineer must quickly determine whether the issue is in **our code**, in the **provider's system**, or in the **infrastructure between them** — and act accordingly.

You are given:

1. System description (simplified)

Our payment platform processes transactions through **payment adapters** — microservices that translate our internal protocol into provider-specific API calls. Each adapter follows a 3-layer architecture:



Supporting infrastructure:

Component	Role
MI (Merchant Interface)	Central orchestrator — routes orders to adapters based on terminal configuration
Terminals Service	Maps merchant terminals to adapter URLs and credentials
Consul	Service discovery — adapters and MI register here
Vault	Stores API keys, signing keys, and adapter credentials
Redis	Caches terminal link lookups (10-min TTL)
RabbitMQ	Async messaging for callbacks and status updates
PostgreSQL	Order persistence (separate instances per adapter)
ELK	Centralized log aggregation
Prometheus/Grafana	Metrics and alerting

Common failure modes:

- External provider degradation (timeouts, 5xx, maintenance windows).
- Credential rotation issues (Vault secret updated but adapter not restarted).
- Connection pool exhaustion under traffic spikes.
- Redis cache containing stale terminal links after configuration change.
- Signing key mismatch (**X-Sign** validation failures) after library upgrade.
- RabbitMQ queue backlog causing delayed callback processing.
- Database connection pool saturation from long-running queries.

2. Examples of past incidents with log snippets

[INC-201] Mass payment failures on adapter cc109 (OPay)

```
2024-11-15 14:23:01.445 [http-nio-10109-exec-12] ERROR c.c.m.client.opay.ClientService
- POST https://api.opay.ng/v3/payments/create failed:
  status=503, body={"message":"Service temporarily unavailable","code":"MAINTENANCE"}
  X-Request-Id: 9f3a2b7c-1d4e-4f8a-b2c1-3e5f7a9b1d3e

2024-11-15 14:23:01.447 [http-nio-10109-exec-12] WARN c.c.m.opay.core.PaymentServiceImpl
- Payment order abc-123 declined: provider returned 503.
  Mapped to: SYSTEM_MALFUNCTION / decline_code=PROVIDER_UNAVAILABLE

2024-11-15 14:23:05.112 [http-nio-10109-exec-08] ERROR c.c.m.client.opay.ClientService
- POST https://api.opay.ng/v3/payments/create failed:
  status=503, body={"message":"Service temporarily unavailable","code":"MAINTENANCE"}
  X-Request-Id: 7b2c3d4e-5f6a-7b8c-9d0e-1f2a3b4c5d6e

... (pattern repeats for 200+ requests over 45 minutes)

2024-11-15 14:23:05.115 [scheduling-1] INFO c.c.m.opay.core.StatusCheckService
- updateOrderStatus: 47 orders stuck in PENDING_IN_BANK, provider still returning 503.
```

[INC-202] Signature validation errors after adapter-commons upgrade

```
2024-12-03 09:15:22.301 [http-nio-10087-exec-03] ERROR c.c.m.payme.api.PaymentController
- Request signature validation failed for order xyz-789.
  Expected X-Sign: sha256=a1b2c3d4e5f6...
  Received X-Sign: sha256=9f8e7d6c5b4a...
  X-Request-Id: 1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d

2024-12-03 09:15:22.305 [http-nio-10087-exec-03] WARN c.c.m.payme.api.PaymentController
- Rejecting request from MI: signature mismatch. Returning 401.

2024-12-03 09:15:30.112 [http-nio-10087-exec-07] ERROR c.c.m.payme.api.PaymentController
- Request signature validation failed for order xyz-790.
  Expected X-Sign: sha256=a1b2c3d4e5f6...
  Received X-Sign: sha256=ff00ee11dd22...
  X-Request-Id: 2b3c4d5e-6f7a-8b9c-0d1e-2f3a4b5c6d7e

2024-12-03 09:17:00.000 [main] INFO c.c.m.payme.Application
- adapter-commons version: 1.7.145-SNAPSHOT (upgraded from 1.7.140)

... (all incoming requests rejected with 401 for the last 2 hours)
```

[INC-203] Payout delays and RabbitMQ queue growth on adapter cc139 (Halopesa)

```
2024-10-22 11:30:15.200 [rabbit-consumer-1] WARN c.c.m.halopesa.core.PayoutServiceImpl
- Payout callback for order pyt-456 received but order still in BEGIN state.
  Expected: PENDING_IN_BANK. Will re-queue for later processing.

2024-10-22 11:30:15.205 [rabbit-consumer-1] INFO c.c.m.halopesa.core.PayoutServiceImpl
```

- Re-queued payout callback to DLQ. Attempt 3 of 5.

2024-10-22 11:32:00.100 [scheduling-1] WARN c.c.m.halopesa.core.StatusCheckService

- Queue depth alert: halopesa.payout.callback queue has 1,247 pending messages.
Consumer throughput: 12 msg/sec. Estimated drain time: ~104 seconds.

2024-10-22 11:35:22.330 [http-nio-10139-exec-01] ERROR c.c.m.client.halopesa.SoapClientService

- SOAP call to https://api.halopesa.com/PayoutService timed out after 30000ms.
X-Request-Id: 3c4d5e6f-7a8b-9c0d-1e2f-3a4b5c6d7e8f

2024-10-22 11:40:00.000 [scheduling-1] ERROR c.c.m.halopesa.core.StatusCheckService

- MongoDB health check failed: com.mongodb.MongoTimeoutException: Timed out after 5000ms waiting for a server matching ReadPreferenceServerSelector.

... (payout processing completely stalled for 25 minutes)

[INC-204] Intermittent "terminal link not found" errors across multiple adapters

2024-09-18 16:45:01.100 [http-nio-10109-exec-05] ERROR c.c.m.opay.core.PaymentServiceImpl

- No terminal link found for terminal=T-4420, paymentMethod=EWALLET, currency=NGN.
Returning FAILED_AUTH with decline=BAD_TERMINAL_CONFIG.
X-Request-Id: 4d5e6f7a-8b9c-0d1e-2f3a-4b5c6d7e8f9a

2024-09-18 16:45:03.200 [http-nio-10131-exec-02] ERROR c.c.m.cobre.core.PaymentServiceImpl

- No terminal link found for terminal=T-7712, paymentMethod=BANK_TRANSFER, currency=COP.
Returning FAILED_AUTH with decline=BAD_TERMINAL_CONFIG.
X-Request-Id: 5e6f7a8b-9c0d-1e2f-3a4b-5c6d7e8f9a0b

2024-09-18 16:45:05.500 [http-nio-10109-exec-11] INFO c.c.m.opay.core.PaymentServiceImpl

- Payment order opay-999 processed successfully.
Terminal=T-5501, terminalLinkId=88432.

2024-09-18 16:46:00.000 [scheduling-1] WARN c.c.m.common.cache.RedisCacheService

- Redis GET terminal-links:T-4420 returned null. Cache miss – calling Terminals Service.

2024-09-18 16:46:00.500 [scheduling-1] ERROR c.c.m.common.cache.RedisCacheService

- Terminals Service returned 404 for terminal=T-4420.
Possible cause: terminal link was deleted or reconfigured.

... (affects ~15% of transactions across 3 adapters for 30 minutes, then self-resolves)

[INC-205] Connection pool exhaustion on adapter cc087 (PayMe)

2024-11-28 20:10:05.100 [http-nio-10087-exec-48] ERROR c.c.m.client.payme.ClientService

- Failed to acquire connection from pool within 5000ms.
Active connections: 50/50, pending requests: 127.
java.util.concurrent.TimeoutException: Timeout waiting for connection from pool
X-Request-Id: 6f7a8b9c-0d1e-2f3a-4b5c-6d7e8f9a0b1c

2024-11-28 20:10:05.102 [http-nio-10087-exec-48] ERROR c.c.m.payme.core.PaymentServiceImpl

- Payment order pay-321 failed: could not reach provider.
Mapped to: SYSTEM_MALFUNCTION / decline_code=CONNECTION_ERROR

2024-11-28 20:10:10.200 [http-nio-10087-exec-49] WARN c.c.m.client.payme.ClientService

- Slow response from https://api.payme.uz/v2/charge: 28,450ms (threshold: 10,000ms).
X-Request-Id: 7a8b9c0d-1e2f-3a4b-5c6d-7e8f9a0b1c2d

2024-11-28 20:10:15.300 [metrics-reporter] INFO c.c.m.common.metrics.AdapterMetrics

- payme.http.requests.active=50, payme.http.requests.queued=134,
payme.http.response.p99=29200ms, payme.http.response.p50=18400ms

2024-11-28 20:10:30.000 [http-nio-10087-exec-50] ERROR c.c.m.client.payme.ClientService

- POST https://api.payme.uz/v2/charge failed:
java.net.SocketTimeoutException: Read timed out (30000ms)
X-Request-Id: 8b9c0d1e-2f3a-4b5c-6d7e-8f9a0b1c2d3e

... (all 50 connections held by slow provider responses, new requests failing immediately)

Your task

Implement a small service (CLI tool, HTTP API, or simple web UI — your choice) that, given log snippets from a payment adapter incident:

1. Classifies the failure, for example:

- "External provider degradation"
- "Internal configuration / credential issue"
- "Infrastructure failure (DB/cache/queue)"
- "Connection pool / resource exhaustion"
- "Routing / terminal configuration issue"

2. Produces a short summary:

- What is happening.
- Which adapter(s) and order types are affected.
- Which architectural layer is the likely source (SDK / Core / API / Infrastructure).
- Severity: **low** / **medium** / **high** / **critical** with reasoning.
- Blast radius: single merchant, single adapter, or platform-wide.

3. Suggests hypotheses and diagnostic steps:

- Up to 3 hypotheses about possible root causes.
- For each hypothesis, 2-3 concrete next steps referencing specific tools and systems (ELK queries, Grafana dashboards, Consul health checks, Vault secret inspection, Redis key lookups, RabbitMQ management UI, etc.).

4. Recommends immediate actions:

- Up to 2 actions the on-call engineer can take right now to mitigate (e.g., restart adapter, flush Redis cache, switch traffic to fallback provider, scale up connection pool).
- For each action, flag the risk level (**safe** / **caution** / **risky**).

Connectivity to log source is out of the scope, you can feed snippets to the app any way you want.

LLM/agent requirements

Mandatory points:

- **Use the LLM as part of an agent architecture**, not as a single "magic" function:
- Explicitly separate stages: parsing log snippets → extracting signals (error types, timestamps, affected components) → enriching with system architecture context → generating structured diagnosis.
- This can be implemented as a custom orchestrator or via any agent framework.
- **Return the result in a machine-readable format:**

```
{
  "incident_id": "string|null",
  "category": "string",
  "summary": {
    "description": "string",
    "affected_adapters": ["string"],
    "affected_order_types": ["string"],
```

```

    "fault_layer": "SDK|Core|API|Infrastructure|External",
    "severity": "low|medium|high|critical",
    "severity_reasoning": "string",
    "blast_radius": "single_merchant|single_adapter|multi_adapter|platform_wide"
  },
  "hypotheses": [
    {
      "title": "string",
      "reasoning": "string",
      "probability": "likely|possible|unlikely",
      "next_steps": [
        {
          "action": "string",
          "tool": "string",
          "detail": "string"
        }
      ]
    }
  ]
},
"immediate_actions": [
  {
    "action": "string",
    "risk": "safe|caution|risky",
    "reasoning": "string"
  }
]
}

```

- **Handle typical model errors:**

- Invalid JSON.
- Hallucinated adapter names or services not in the system description.
- Hypotheses that contradict the log evidence.
- Missing fields or unexpected severity values.

- **Show a recovery strategy:**

- For example, validate adapter names against the known system description; if the model invents one, re-prompt.
- Validate that `fault_layer` is one of the 5 allowed values.
- Ensure `next_steps` reference real tools from the infrastructure table (ELK, Grafana, Consul, Vault, Redis, RabbitMQ, PostgreSQL).

Technical conditions

- Programming language and stack — your choice.
- You may use:
 - Any public LLM API (OpenAI, Anthropic, Gemini, etc.) or a local model.
 - Any LLM/agent libraries/frameworks (LangChain, Semantic Kernel, etc.).
- Important:
 - Move configuration (API keys, endpoints) to external settings/environment variables.
 - Keep the solution compact and readable — no need for heavy enterprise boilerplate.

What to submit

1. A link to the repository (GitHub/GitLab/Bitbucket, etc.) with your code.
 2. A **README** briefly describing:
 - How to run the project (minimal steps).
 - How the agent is structured at a high level (1-2 paragraphs or a simple step-by-step description).
 - How you tested the behavior: run it on at least 3 of the 5 incidents above and describe what kind of outputs you expected vs. what you got.
 3. (Optional, but appreciated) A short note on trade-offs:
 - What you simplified to fit into ~3 hours.
 - What you would do differently with more time (e.g., integration with real ELK/Grafana APIs, historical incident correlation, runbook auto-generation).
-

Mini example of expected behavior (non-canonical)

Input:

```
2024-11-15 14:23:01.445 [http-nio-10109-exec-12] ERROR c.c.m.client.opay.ClientService
- POST https://api.opay.ng/v3/payments/create failed:
  status=503, body={"message":"Service temporarily unavailable","code":"MAINTENANCE"}
  X-Request-Id: 9f3a2b7c-1d4e-4f8a-b2c1-3e5f7a9b1d3e

2024-11-15 14:23:05.112 [http-nio-10109-exec-08] ERROR c.c.m.client.opay.ClientService
- POST https://api.opay.ng/v3/payments/create failed:
  status=503, body={"message":"Service temporarily unavailable","code":"MAINTENANCE"}

... (repeating for 45 minutes)
```

Example of a possible (schematic) answer:

```
{
  "incident_id": "INC-201",
  "category": "External provider degradation",
  "summary": {
    "description": "The OPay provider is returning 503 with a MAINTENANCE code for all payment creation requests.",
    "affected_adapters": ["cc109 (OPay)"],
    "affected_order_types": ["PAYMENT"],
    "fault_layer": "External",
    "severity": "high",
    "severity_reasoning": "All payment transactions through OPay are failing. The provider is in maintenance mode.",
    "blast_radius": "single_adapter"
  },
  "hypotheses": [
    {
      "title": "Planned or unplanned maintenance on OPay side",
      "reasoning": "The 503 response includes a MAINTENANCE code, suggesting the provider is deliberately offline.",
      "probability": "likely",
      "next_steps": [
        {
          "action": "Check OPay status page and recent communications",
          "tool": "Browser / Email",
          "detail": "Look for maintenance notifications from OPay sent in the last 24 hours. Check https://status.opay.ng"
        },
        {
          "action": "Verify the issue is not on our side",
          "tool": "ELK",
          "detail": "Query: service=cc109 AND level=ERROR AND timestamp>[2024-11-15T14:00:00Z]. Confirm all errors are from OPay."
        },
        {
          "action": "Check if other merchants on OPay are affected",
          "tool": "Grafana",
          "detail": "Dashboard: adapter-cc109-overview. Check if the failure rate is 100% across all terminals or orders."
        }
      ]
    }
  ]
}
```

```

    ]
  }
],
"immediate_actions": [
  {
    "action": "If available, temporarily reroute OPay traffic to a fallback provider via terminal link reconfiguration",
    "risk": "caution",
    "reasoning": "Requires coordination with the merchant and may not be possible for all payment methods. But w
  },
  {
    "action": "Enable circuit breaker to fast-fail OPay requests and return a user-friendly error instead of waiti
    "risk": "safe",
    "reasoning": "Reduces load on the adapter and improves customer experience by failing fast with a clear messa
  }
]
}

```